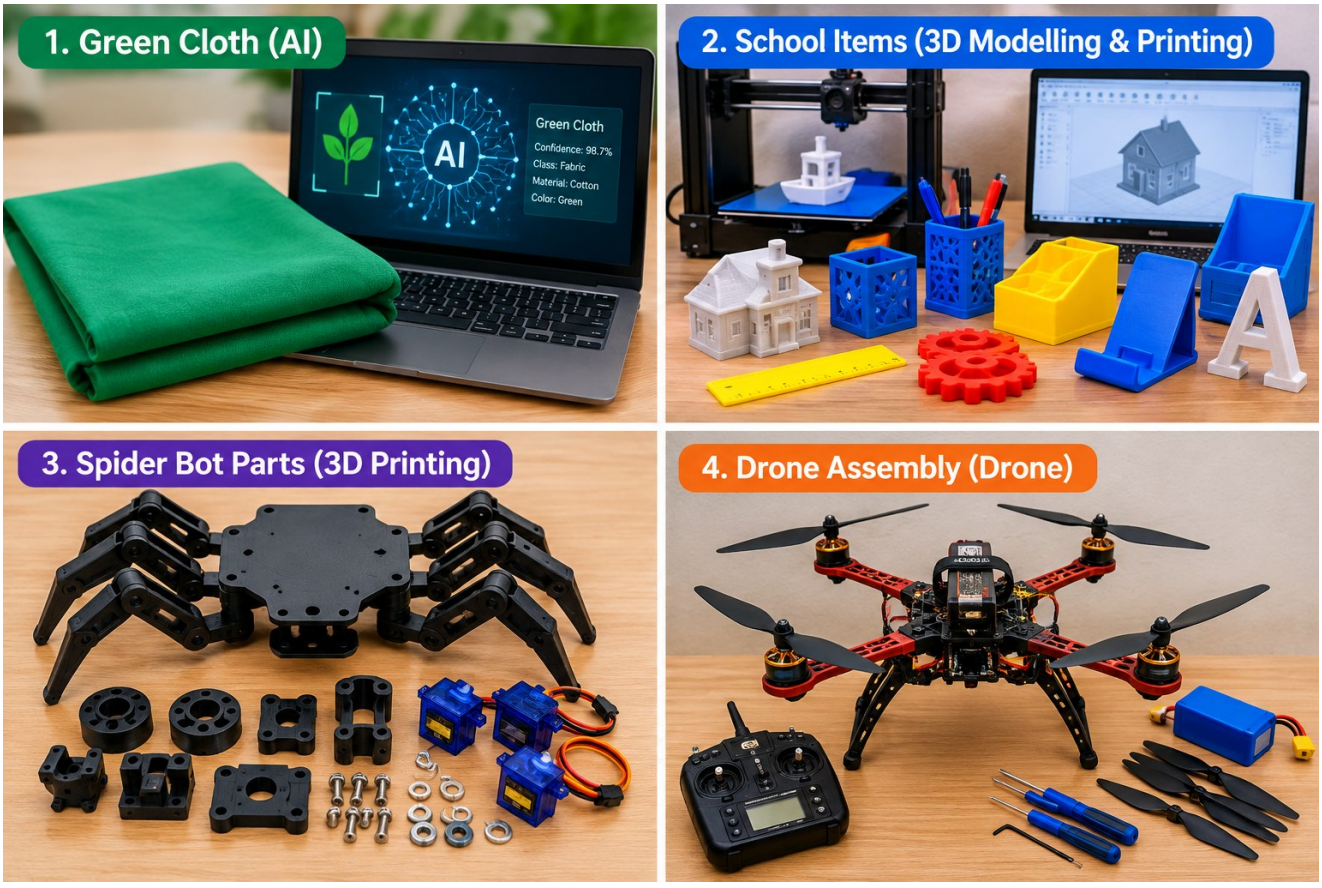
TERRAMIND EDTECH

Learn by Making

AI, Arduino, 3D Printing, Robots and Drones

A simple teaching guide for Classes 6, 7, 8 and 9



Our four hands-on projects: Green Cloth (AI), School Items (3D printing), Spider Bot and Drone

What is inside? Every lesson is written the way a teacher would explain it in class: first the big idea, then the steps, then the code explained line by line, then an activity and a quiz with answers.

Class 6	Class 7	Class 8	Class 9
LEDs, traffic light, how a computer sees color	Buttons, sensors, finding green with HSV	Invisible cloth: background replacement, 3D printing	Final project, testing, spider bot and drone

The Big Picture

Our course has three strands. Students meet each strand again every year, but the ideas get deeper as they grow.

Class	Arduino (electronics)	Python + OpenCV (AI vision)	Maker projects
6	LED blink, Traffic light, Running lights	Live camera and color lab: click a pixel and read BGR / HSV	Safety and tools
7	Push button, Night lamp (LDR), Distance warning	Find green with HSV sliders: mask and highlight	3D modelling basics: school items
8	Revision: combine sensors	Invisible cloth: save the background and replace green	3D printing: design, slice, print
9	Challenge: build your own	Final project: controls, FPS, screenshots, testing report	Spider bot and drone assembly

How to use this guide
Teach in the order given. Each class has a banner page in its own color. Blue = Class 6, Green = Class 7, Orange = Class 8, Purple = Class 9. Sections called **What the teacher says** are short scripts you can read aloud.

Words every student should know

Word	Simple meaning	Everyday example
Pixel	One tiny colored dot of a picture	A photo is made of thousands of colored dots, like a mosaic
Frame	One single picture from the camera	A video is many frames shown quickly, like a flip-book
BGR	OpenCV stores colors as Blue, Green, Red numbers (0 to 255)	(0, 255, 0) means no blue, full green, no red = pure green
HSV	Hue (which color), Saturation (how strong), Value (how bright)	Same green: bright and strong, or pale, or dark
Mask	A black and white picture: white means 'yes, this is green'	A stencil that lets paint through only where you cut holes
Library	A box of ready-made tools we import	cv2 is the camera and picture toolbox
Variable	A named box that stores something	background stores the empty-room picture
Loop	Instructions that repeat again and again	while True keeps reading the camera
Function	A small named recipe we can reuse	capture_background() does the same job every time
FPS	Frames Per Second: how smooth the video is	30 FPS looks smooth, 5 FPS looks jumpy

CLASS 6

Meet Arduino and the Science of Color

Blink LEDs, build a traffic light, then discover how a computer sees color through a webcam.

Part 1: Arduino Nano components

Arduino Nano is a tiny computer. We give it a program, and it switches LEDs, reads buttons and measures light and distance. Every group gets one Arduino Nano, one breadboard, jumper wires, LEDs, resistors, a push button, a potentiometer, a buzzer, an HC-SR04 distance sensor and an LDR light sensor.

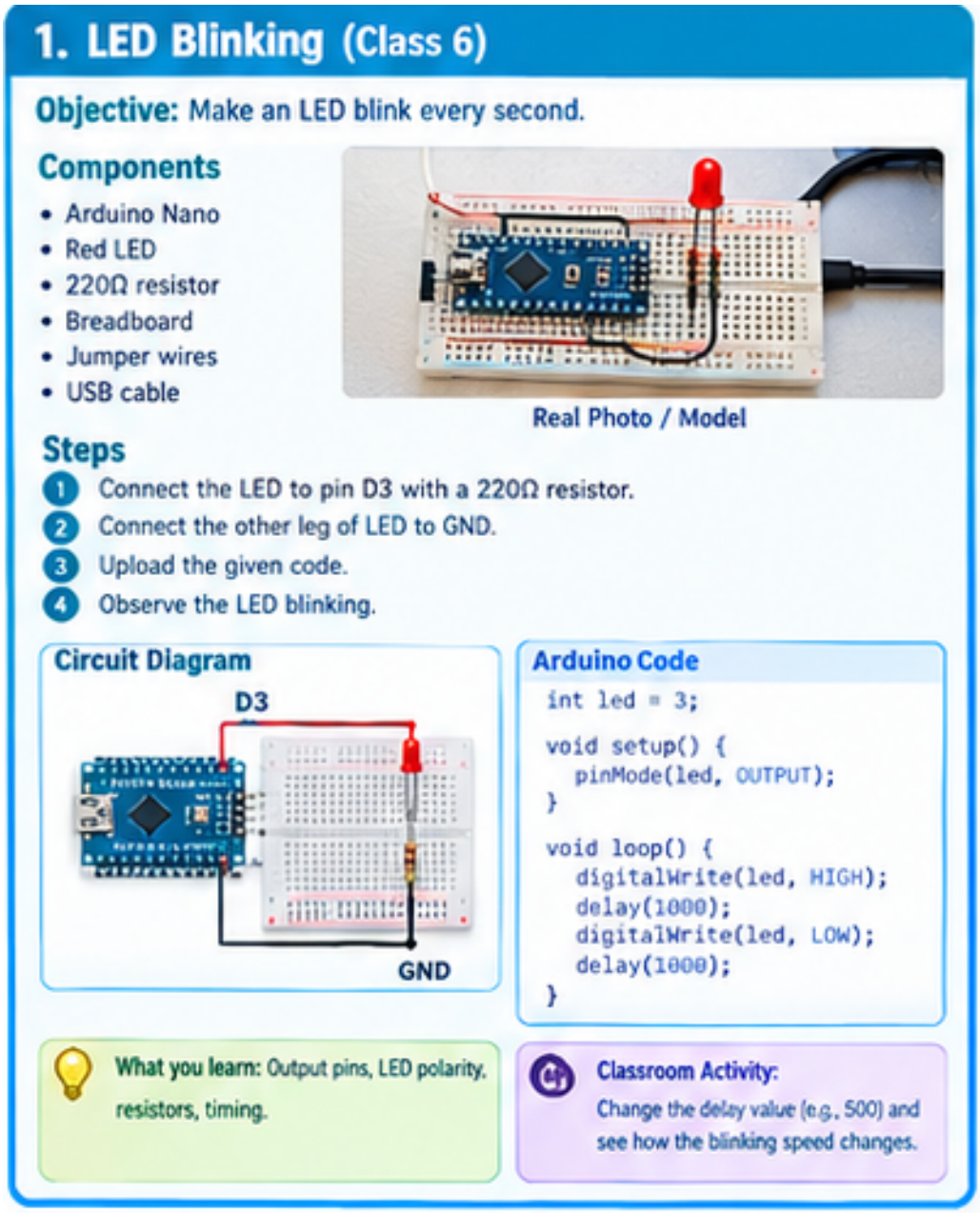


Components used in most Arduino projects

Safety first
Use only the 5V USB supply. Always put a resistor with an LED, otherwise the LED can burn out. Never connect motors directly to Arduino pins. Check the pinout of your Nano before connecting wires.

Part 2: Project 1 - LED Blinking

Goal: make one LED blink every second. This is the 'Hello World' of electronics.



Project 1 from the Arduino guide: wiring, circuit diagram and code

Steps

- Connect the LED's long leg (+) to pin **D3** through a 220 ohm resistor.
- Connect the LED's short leg (-) to **GND**.
- Upload the code.
- Watch the LED blink.

Code explained

Code	What it means (in simple words)
`int led = 3;`	We give pin number 3 a friendly name, 'led'.
`void setup() {`	setup runs ONCE when the board starts.
`  pinMode(led, OUTPUT);`	Tell Arduino: pin 3 will send power OUT to something.
`void loop() {`	loop runs again and again, forever.
`  digitalWrite(led, HIGH);`	Switch pin 3 ON (5 volts). LED lights up.

Code	What it means (in simple words)
`delay(1000);`	Wait 1000 milliseconds = 1 second.
`digitalWrite(led, LOW);`	Switch pin 3 OFF. LED goes dark.
`delay(1000);`	Wait 1 second again, then loop starts over.

What the teacher says in class

Arduino is like a very obedient robot. It does exactly what we write, in order. HIGH means ON, LOW means OFF, and delay means wait.

Classroom activity

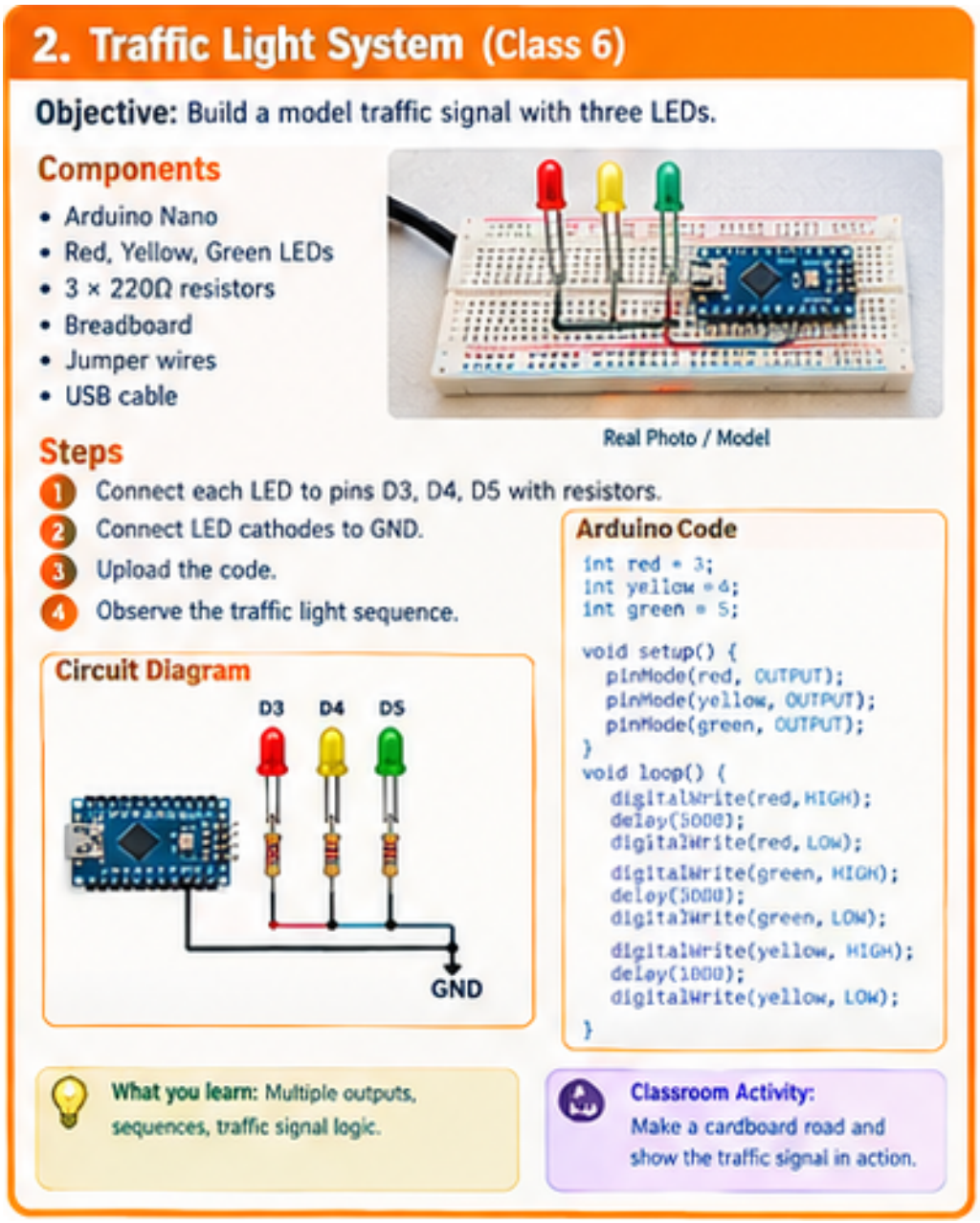
Change delay(1000) to delay(500) and then delay(100). What happens to the blinking speed? Which value makes it look as if the LED is always on?

What students learn

Output pins, LED polarity (long leg is +), why resistors protect LEDs, and timing.

Part 3: Project 2 - Traffic Light System

Goal: build a model traffic signal with red, yellow and green LEDs.



Project 2: traffic light wiring and code

- Red LED to D3, Yellow LED to D4, Green LED to D5, each with its own 220 ohm resistor.
- All LED short legs (cathodes) go to GND.
- Upload the code and watch the sequence.

Note for the teacher: the picture shows the code with red = 3, yellow = 4, green = 5. Make sure students' wiring matches their code.

```
int red = 3;  int yellow = 4;  int green = 5;

void setup() {
  pinMode(red, OUTPUT);  pinMode(yellow, OUTPUT);  pinMode(green, OUTPUT);
}
void loop() {
  digitalWrite(red, HIGH);    delay(3000);  digitalWrite(red, LOW);     // STOP for 3 seconds
  digitalWrite(green, HIGH);  delay(3000);  digitalWrite(green, LOW);   // GO for 3 seconds
  digitalWrite(yellow, HIGH); delay(1000);  digitalWrite(yellow, LOW);  // GET READY for 1 second
}
```

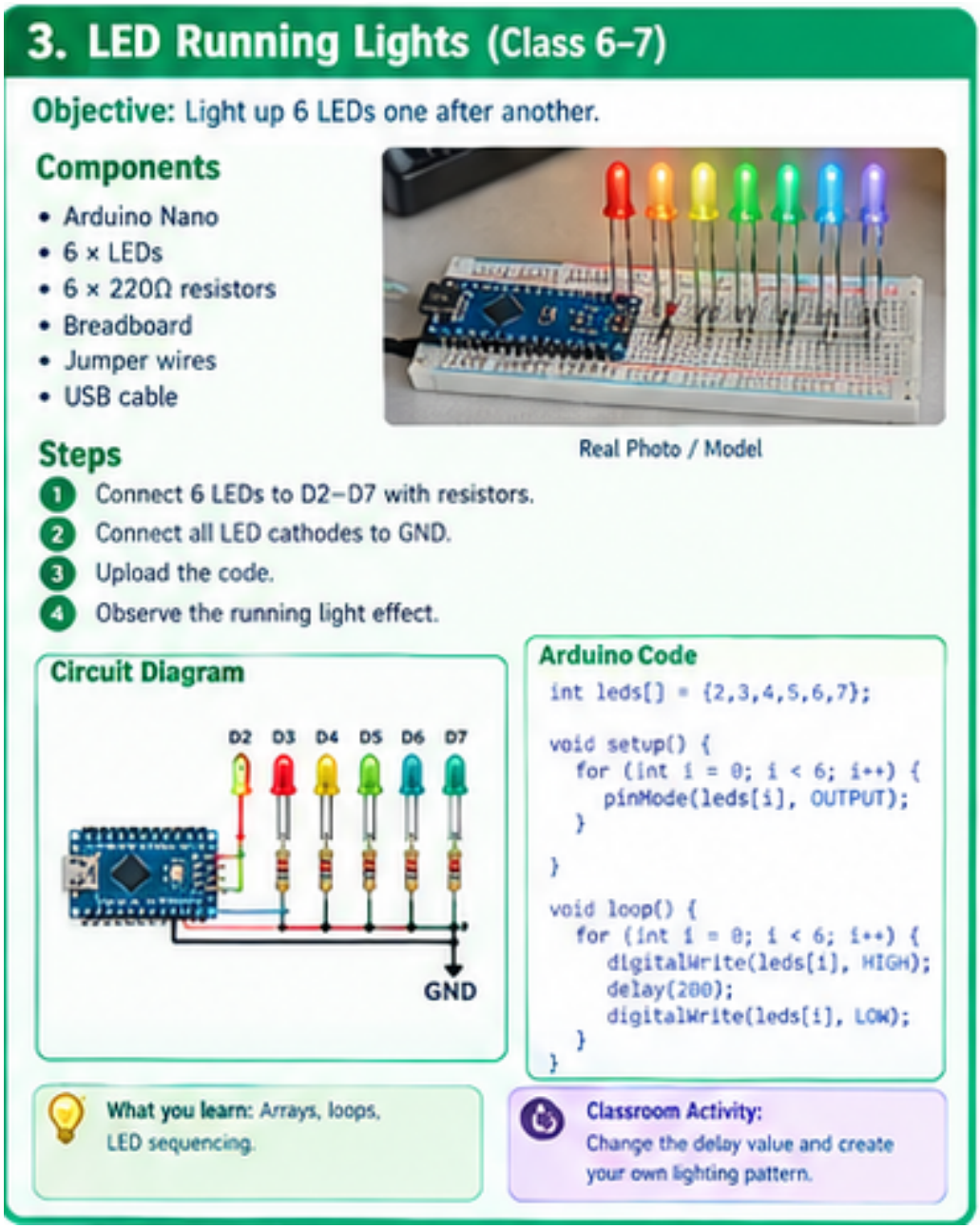
How it works: three LEDs are three outputs. In loop() we switch on one LED, wait, switch it off, then move to the next. The order of lines is the order of the lights. That is called a **sequence**.

Classroom activity
Make a cardboard road with a toy car. Run the traffic light and let a 'car' stop and go. Then change the timings so the green light is longer.

What students learn
Multiple outputs, sequences, traffic-signal logic.

Part 4: Project 3 - LED Running Lights

Goal: light up 6 LEDs one after another, like a Diwali string light chasing across.



Project 3: running lights

```
int leds[] = {2,3,4,5,6,7};          // a list (array) of 6 pin numbers

void setup() {
  for (int i = 0; i < 6; i++) {      // repeat 6 times: i = 0,1,2,3,4,5
    pinMode(leds[i], OUTPUT);        // make every pin an output
  }
}
void loop() {
  for (int i = 0; i < 6; i++) {
    digitalWrite(leds[i], HIGH);     // turn LED number i ON
    delay(200);                      // wait a little
    digitalWrite(leds[i], LOW);      // turn it OFF, then next LED
  }
}
```

New idea	Simple meaning
Array int leds[] = {2,3,4,5,6,7}	One box with six compartments. leds[0] is 2, leds[1] is 3, and so on. Counting starts from 0!
for loop	A counter that repeats code. 'for i from 0 to 5' means do it 6 times, with i changing each time.
i++	Add 1 to i. It is a short way of writing i = i + 1.

Classroom activity
Change the delay to 100 and then 500. Then design your own pattern: for example, light the LEDs from right to left, or light two at a time.

Part 5: Python + OpenCV - How does a computer see color?



Our Green Cloth project: a camera, a laptop and a green cloth

Our eyes see 'green'. A computer sees only **numbers**. In this lesson the webcam shows live video. When we click on any part of the video, the program tells us the exact numbers of that pixel, and the name of the color.

Big idea
Every pixel is 3 numbers: how much Blue, how much Green and how much Red (each from 0 to 255). OpenCV uses the order **B, G, R**. The computer can also change those numbers into **H, S, V** (Hue, Saturation, Value), which is much easier for finding a color.

Setup: install the tools (once)

```
pip install opencv-python numpy
```

Step 1: import and open the camera

Code	What it means (in simple words)
`import cv2`	Bring in OpenCV: our toolbox for camera and pictures.
`cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)`	Open camera number 0 (the first camera). CAP_DSHOW is a Windows setting that avoids camera errors.
`if not cap.isOpened():`	Did the camera fail to open? Then...
`    cap = cv2.VideoCapture(0)`	...try again the normal way.
`    print('Webcam not found...')` `    raise SystemExit`	If it still fails, show a message and stop the program.

Step 2: variables that remember what happened

Code	What it means (in simple words)
`info_text = 'Click anywhere...'`	The message shown at the top of the screen.
`color_name = ''`	Will store the name of the color, like 'Green'.
`clicked_bgr = None`	Will store the BGR numbers of the clicked pixel. None = nothing clicked yet.
`click_pos = None`	Will store where on the screen we clicked.
`state = {'frame': None}`	A small notebook that holds the latest camera picture so the click function can look at it.

Step 3: the function that names the color

```
def get_color_name(h, s, v):
    if v < 40:   return "Black / Very Dark"      # not enough light
    if s < 30:   return "White" if v > 200 else "Gray"   # no color strength
    if 35 <= h <= 85:  return "Green (Green Screen Range)"
    elif 85 < h <= 135: return "Blue"
    ...
```

Test	Meaning
v < 40	Value (brightness) is very low, so it is nearly black. We cannot trust the color.
s < 30	Saturation is very low: the color is washed out. If bright it is white, otherwise gray.
35 <= h <= 85	Hue between 35 and 85 on OpenCV's color wheel = green. This is the range we will use for the cloth.
other h ranges	Blue, purple, red, orange / skin tone and yellow are checked in the same way.

Teacher tip
OpenCV hue goes from 0 to 179 (not 360). Draw a rainbow strip on the board: red 0, yellow 30, green 60, cyan 90, blue 120, purple 150.

Step 4: the mouse-click function

Code	What it means (in simple words)
`def on_click(event, x, y, flags, param):`	OpenCV calls this function every time the mouse does something. x and y = where the mouse is.
`if event == cv2.EVENT_LBUTTONDOWN`	Only react to a LEFT mouse button press.
`b, g, r = [int(v) for v in frame[y, x]]`	Look at the pixel at position (x, y) and read its Blue, Green, Red numbers.
`hsv = cv2.cvtColor(...COLOR_BGR2HSV)`	Convert that one pixel into Hue, Saturation, Value.
`color_name = get_color_name(h, s, v)`	Ask our function: what is this color called?
`info_text = f'BGR=({b},{g},{r}) ...'`	Make a text message with the numbers (an f-string puts variables inside text).
`cv2.setMouseCallback(window, on_click)`	Connect the window to our function, so clicks are noticed.

Step 5: the main loop (draw and show)

Code	What it means (in simple words)
`while True:`	Repeat forever, once for every camera frame.
`ok, frame = cap.read()`	Take a picture. ok = did it work? frame = the picture.
`state['frame'] = frame.copy()`	Save a copy for the click function.
`cv2.rectangle(frame, ...)`	Draw a dark banner at the top, and a small square filled with the clicked color (the swatch).
`cv2.putText(frame, ...)`	Write the color name and the numbers on the picture.
`cv2.circle / cv2.line`	Draw a circle and a cross where we clicked.
`cv2.imshow('Class 6 - Live Camera', frame)`	Show the picture in a window.
`cv2.waitKey(1) & 0xFF == ord('q')`	Check if the key Q was pressed. If yes, break (leave the loop).
`cap.release()` `cv2.destroyAllWindows()`	Switch off the camera and close all windows. Always clean up!

Classroom activity

Lighting experiment. Click on the green cloth in bright light, then switch off a light or cover the window and click again. Write the HSV numbers in a table. Which number changed most? (Answer: usually V, the brightness.) This is why lighting matters for AI cameras.

What the teacher says in class

The computer is not clever like us. It does not 'see' green; it checks numbers. Today we find out what numbers green has.

Quick Quiz (with answers)

#	Question	Answer
1	How many numbers make one pixel's color?	Three (B, G, R).
2	What does 'void setup()' do in Arduino?	Runs once when the board starts.
3	Why do we use a resistor with an LED?	To protect the LED from too much current.
4	What does HIGH mean in digitalWrite?	Switch the pin ON (5 volts).
5	In the color lab, which letter in HSV shows brightness?	V (Value).

#	Question	Answer
6	Which key closes the camera program?	Q.

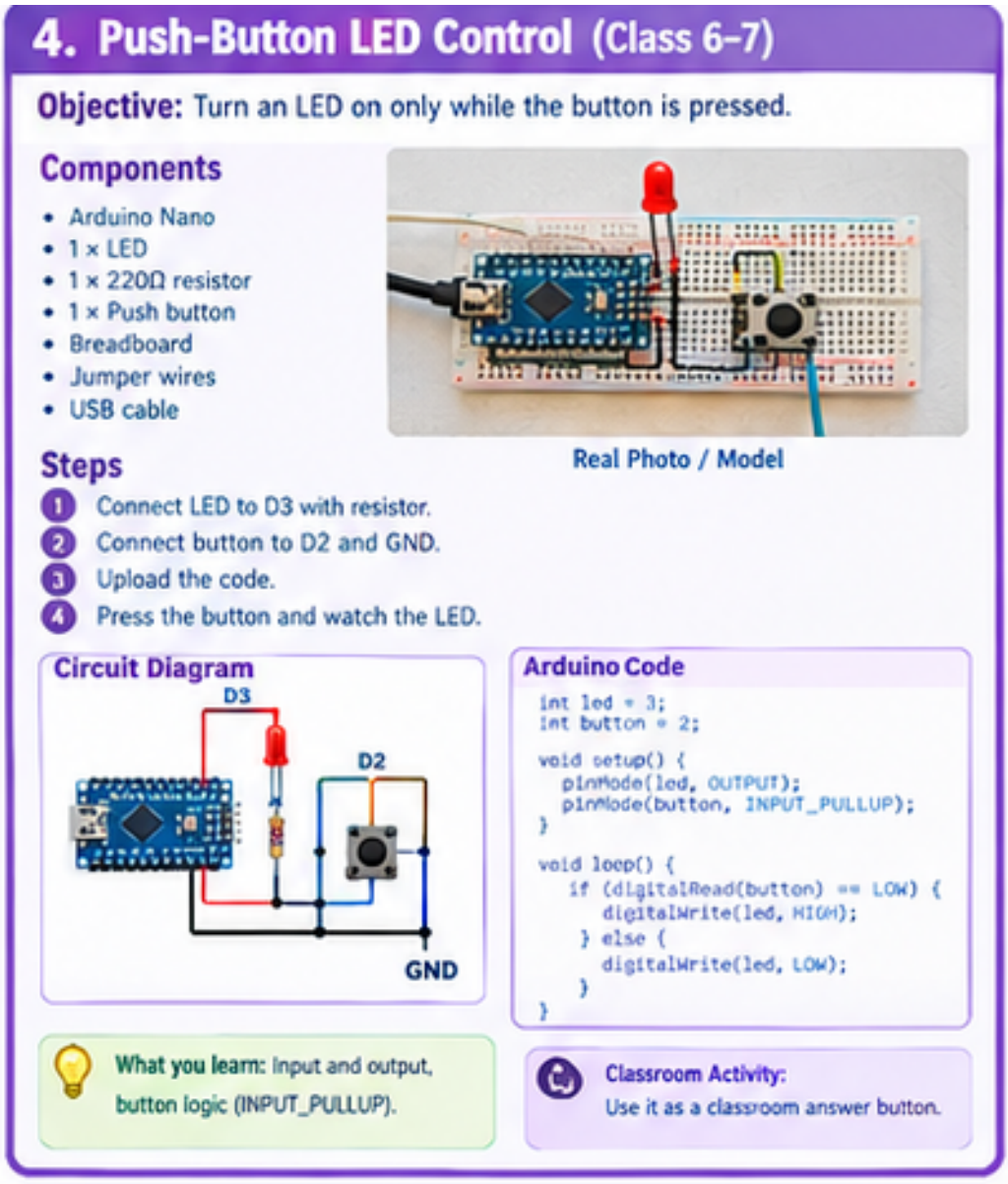
CLASS 7

Sensors, Buttons and Finding Green

Arduino learns to sense the world; Python learns to find the green cloth using an HSV mask.

Part 1: Project 4 - Push-Button LED Control

Goal: turn an LED on only while the button is pressed. Now Arduino has an **input** as well as an output.



Project 4: push button and LED

```
int led = 3;  int button = 2;

void setup() {
  pinMode(led, OUTPUT);
  pinMode(button, INPUT_PULLUP);        // button pin is an input with a built-in pull-up
}
void loop() {
  if (digitalRead(button) == LOW) {     // LOW means the button IS pressed
    digitalWrite(led, HIGH);            // LED on
  } else {
    digitalWrite(led, LOW);             // LED off
  }
}
```

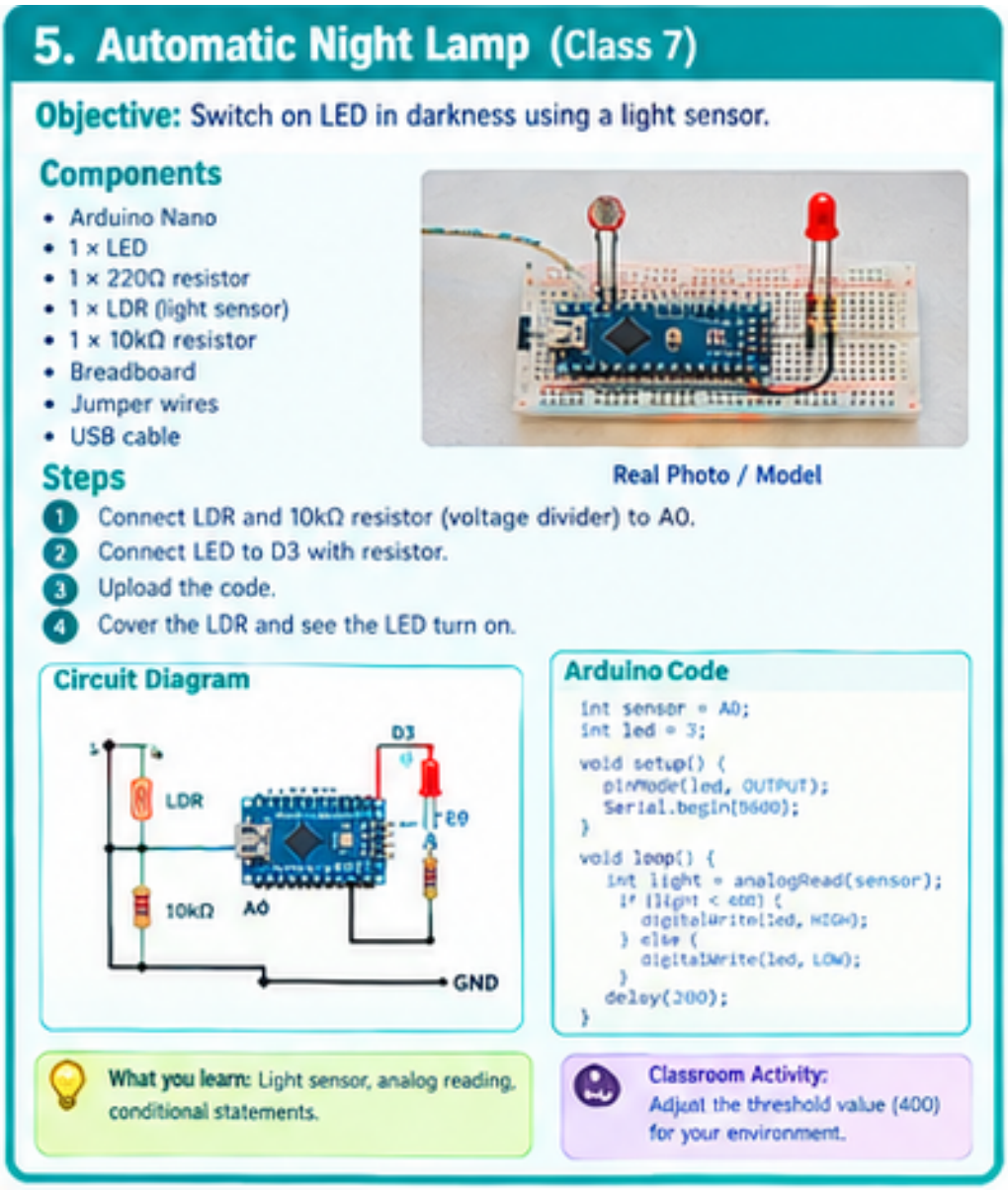
Idea	Simple meaning
INPUT_PULLUP	The pin is gently held at HIGH by the Arduino. The button connects it to GND, so pressing makes it LOW.
digitalRead(button)	Ask: is the pin HIGH or LOW right now?
if ... else	A decision. IF the button is pressed, light the LED, ELSE (otherwise) switch it off.
== (two equal signs)	Means 'is equal to?' (a question). A single = means 'store this value'.

Wiring: LED to D3 with a resistor; button between D2 and GND. Because of the pull-up, we do not need an extra resistor for the button.

Classroom activity
Use the button as a classroom answer button: whoever presses first lights their LED. Discuss: why does the LED go off when you let go?

Part 2: Project 5 - Automatic Night Lamp

Goal: an LED that switches on by itself when it gets dark, like a street light.



Project 5: LDR (light sensor) night lamp

- An **LDR** (light dependent resistor) changes its resistance with light: bright = low resistance, dark = high resistance.
- We connect it with a 10k ohm resistor to make a **voltage divider**; the middle point goes to pin **A0**.
- LED goes to D3 with a 220 ohm resistor.

```
int sensor = A0;  int led = 3;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);               // start sending numbers to the computer screen
}
void loop() {
  int light = analogRead(sensor);   // read a number from 0 to 1023
  if (light < 400) {                // small number = dark room (with this wiring)
    digitalWrite(led, HIGH);        // lamp ON
  } else {
    digitalWrite(led, LOW);         // lamp OFF
  }
  delay(200);
}
```

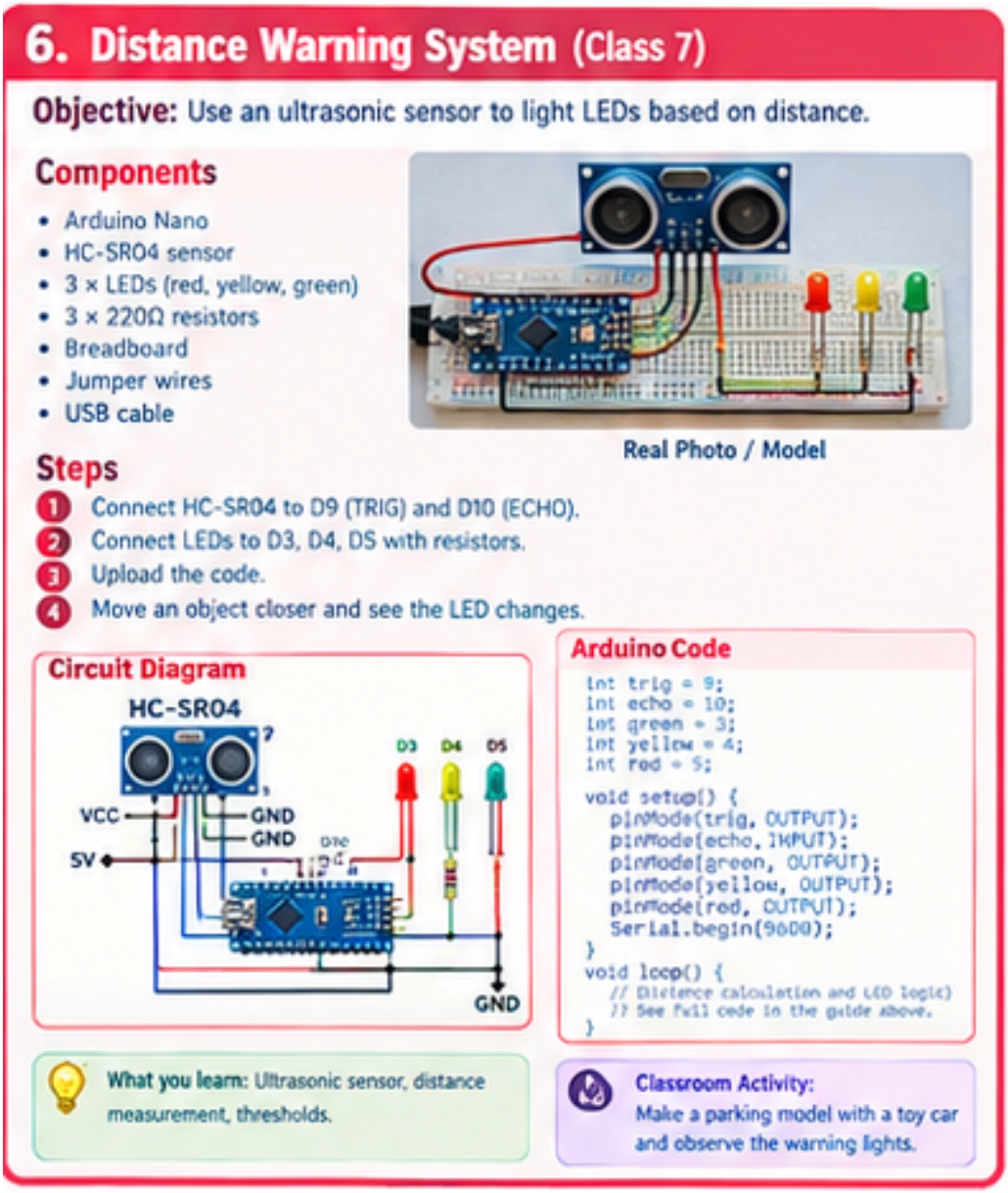
Idea	Simple meaning
analogRead	Reads a smooth number from 0 to 1023, not just ON/OFF. 0 = 0 volts, 1023 = 5 volts.
Serial.begin(9600)	Opens a message line to the computer. Use the Serial Monitor to see the light numbers.
Threshold (400)	The dividing line between 'dark' and 'bright'. Change it to suit your room.

Teacher tip
If your lamp works the opposite way (ON in light), just swap the LDR and the 10k resistor, or change the < sign to >. Open the Serial Monitor first to look at the numbers in light and in dark, then choose the threshold.

Classroom activity
Open the Serial Monitor. Write down the number when the LDR is uncovered and when covered. Choose your own threshold halfway between them.

Part 3: Project 6 - Distance Warning System

Goal: use an ultrasonic sensor (HC-SR04) to light green, yellow or red LEDs depending on how close an object is - like a car's parking sensor.



Project 6: distance warning with HC-SR04

How the sensor works: it sends a sound too high for us to hear. The sound hits an object and comes back as an echo. The time the echo takes tells us the distance, just like a bat finds its way in the dark.

Wiring: VCC to 5V, GND to GND, TRIG to D9, ECHO to D10. Green LED D3, Yellow LED D4, Red LED D5 (each with a resistor).

```
int trig = 9; int echo = 10;
int green = 3; int yellow = 4; int red = 5;

void setup() {
  pinMode(trig, OUTPUT);  pinMode(echo, INPUT);
  pinMode(green, OUTPUT); pinMode(yellow, OUTPUT); pinMode(red, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  digitalWrite(trig, LOW);  delayMicroseconds(2);
  digitalWrite(trig, HIGH); delayMicroseconds(10);   // send a short sound pulse
  digitalWrite(trig, LOW);
  long time = pulseIn(echo, HIGH);                   // how long until the echo returns?
  int distance = time * 0.034 / 2;                   // convert to centimetres

  digitalWrite(green, LOW); digitalWrite(yellow, LOW); digitalWrite(red, LOW);
  if (distance < 10)      digitalWrite(red, HIGH);     // very close: DANGER
  else if (distance < 30) digitalWrite(yellow, HIGH);  // getting close: CAREFUL
  else                    digitalWrite(green, HIGH);   // far away: SAFE
  Serial.println(distance);
  delay(200);
}
```

Why divide by 2? The sound travels to the object AND back, so the time counts the distance twice.
Why 0.034? Sound travels about 0.034 centimetres every microsecond.

Teacher tip
The full code is not printed in the picture ('see full code in guide'), so the code above is a standard working version. Check the pins match your wiring before uploading.

Classroom activity
Make a parking model with a toy car. Move a cardboard 'wall' closer and watch the lights change. Try new distances for red and yellow.

Part 4: Python - Finding green with an HSV mask

In Class 6 we learned that colors are numbers. Today the computer will look at **every pixel** in every frame and decide: is it green or not? The answer is a **mask**.



The green cloth we want the computer to find

Three windows will open
1. Original - what the camera sees. **2. Mask** - white = green found, black = not green. **3. Highlight** - the green pixels are painted red. A fourth small window called **Sliders** lets us change the range live.

Why HSV and not BGR?

A green cloth in shadow has very different B, G, R numbers from the same cloth in sunlight. But its **Hue** stays almost the same. So searching by Hue is much more reliable. That is why we convert BGR to HSV first.

The sliders

Code	What it means (in simple words)
`def nothing(x):` `  pass`	A function that does nothing. OpenCV sliders must have one, but we do not need it.
`cv2.namedWindow('Sliders')`	Make a window to hold our sliders.
`createTrackbar('H low', 'Sliders', 35, 179, nothing)`	Make a slider called H low. Starts at 35. Maximum 179.
`createTrackbar('H high', ..., 85, 179, ...)`	Slider for the highest hue accepted. Starts at 85.
`createTrackbar('S low', ..., 80, 255, ...)`	Lowest saturation accepted: removes gray and white.
`createTrackbar('V low', ..., 50, 255, ...)`	Lowest brightness accepted: removes dark pixels.

The main loop, step by step

Code	What it means (in simple words)
`hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`	STEP 1: change the picture from BGR to HSV.
`h_low = cv2.getTrackbarPos('H low', 'Sliders')`	STEP 2: read where each slider is right now (4 lines like this).
`lower_green = np.array([h_low, s_low, v_low])`	The smallest H, S, V we accept. np.array makes a list of numbers that OpenCV understands.
`upper_green = np.array([h_high, 255, 255])`	The biggest H, S, V we accept (S and V upper limits stay at 255).
`mask = cv2.inRange(hsv, lower_green, upper_green)`	STEP 3: for every pixel, is it between lower and upper? Yes = white (255), No = black (0).
`highlight = frame.copy()`	Make a copy so we do not damage the original picture.
`highlight[mask > 0] = (0, 0, 255)`	STEP 4: wherever the mask is white, paint that pixel RED (0 blue, 0 green, 255 red).
`cv2.imshow('Original' / 'Mask' / 'Highlight', ...)`	Show the three windows.

Classroom activity
Slider challenge: (1) Move 'S low' to 0: what extra things turn white? (2) Move 'V low' to 0: do dark areas become white? (3) Narrow H low and H high until only the cloth is white. Write your best values on the board.

What the teacher says in class
The sliders are like a sieve. We tell the computer: only let colors through that are green enough, strong enough and bright enough.

Quick Quiz (with answers)

#	Question	Answer
1	What does the LDR sensor measure?	The amount of light.
2	Why does the button use INPUT_PULLUP?	So the pin is HIGH normally and LOW when the button is pressed.
3	What does analogRead return?	A number from 0 to 1023.
4	Why do we divide the echo time by 2?	The sound goes to the object and comes back.
5	In the mask, what does a white pixel mean?	That pixel is inside the green range.
6	Which function creates the mask?	cv2.inRange()

CLASS 8

The Invisible Cloth and 3D Printing

Replace the green cloth with the saved background, and learn how ideas become 3D printed objects.

Part 1: How does the cloth become invisible?

Imagine a magic trick: we take a photo of the empty room. Later, wherever the camera sees green cloth, the computer **shows the photo of the empty room instead**. The cloth seems to disappear. This is exactly how movie green screens work.

Step	What we do	What the computer does
1	Press **B** while the cloth and people are OUT of view	Takes 30 pictures and averages them into one clean background
2	Bring the cloth into view	Finds green pixels and makes a mask
3	Watch the screen	Wherever the mask is white, shows the saved background; elsewhere shows the live camera

Golden rule
Keep the camera **perfectly still** after pressing B. If the camera moves, the saved background will not line up with the live picture and the trick fails.

Part 2: The code, part by part

A. Settings and the cleaning tool

Code	What it means (in simple words)
`background = None`	No background saved yet. None means 'empty'.
`kernel = np.ones((3, 3), np.uint8)`	A small 3 x 3 square of ones. It is the 'brush' OpenCV uses to clean the mask. uint8 = whole numbers 0 to 255.
`LOWER_GREEN = np.array([30, 30, 15])`	Smallest H, S, V counted as green. Low S and V catch dark folds and shadows of the cloth.
`UPPER_GREEN = np.array([90, 255, 255])`	Largest H, S, V counted as green.

B. Capturing the background

Code	What it means (in simple words)
`def capture_background(cap, frames=30):`	Make a function. It takes the camera and how many pictures to take (30).
`collected = []`	An empty list to keep the pictures.
`for _ in range(frames):`	Repeat 30 times. The _ means we do not need the counter.
`ok, f = cap.read()`	Take one picture.
`if ok: collected.append(f.astype(np.float32))`	If it worked, save it. float32 allows decimal numbers so that averaging is accurate.

Code	What it means (in simple words)
`return np.mean(collected, axis=0).astype(np.uint8)`	Average all pictures pixel by pixel, change back to normal picture numbers, and give it to the main program.

Why 30 pictures, not 1?
A webcam picture has a little random 'noise' (tiny flickers). Averaging 30 pictures makes the noise cancel out, like asking 30 friends and taking the average answer.

C. The main loop

Code	What it means (in simple words)
`if background is None:`	Have we saved a background yet? No...
`    cv2.putText(frame, msg, (10, 30), ...)`	...write 'Press b to capture the empty background' in yellow (0,255,255 in BGR), at position (10, 30).
`else:`	Yes, we have a background, so do the magic.
`hsv = cv2.cvtColor(...)`	Change to HSV.
`mask = cv2.inRange(hsv, LOWER_GREEN, UPPER_GREEN)`	White where green.

D. Cleaning the mask (three tricks)

Code	Name	Simple explanation
`cv2.morphologyEx(mask, MORPH_OPEN, kernel, iterations=2)`	Opening	Erases tiny white dots (noise) like using an eraser on small specks.
`cv2.dilate(mask, kernel, iterations=1)`	Dilation	Makes white areas a little bigger so no green edge is left behind.
`cv2.GaussianBlur(mask, (5, 5), 0)`	Blur	Softens the edges so the blend looks smooth instead of sharp and jagged.

E. The blending formula (the heart of the project)

Code	What it means (in simple words)
`mask_3ch = cv2.merge([mask, mask, mask])`	The mask has one layer (gray). Pictures have three (B, G, R). We copy the mask three times so they match.
`.astype(np.float32) / 255.0`	Change the numbers from 0..255 into 0..1. Now 0 = 'use camera', 1 = 'use background'.
`result = (background * mask_3ch + frame * (1 - mask_3ch))`	Mix the two pictures using the mask.
`.astype(np.uint8)`	Change back to normal picture numbers.

Try the math with numbers
Suppose one pixel is 200 in the background picture and 50 in the live picture.
Mask = 1 (green cloth): 200 x 1 + 50 x 0 = **200** -> shows the background.
Mask = 0 (a face): 200 x 0 + 50 x 1 = **50** -> shows the live picture.
Mask = 0.5 (soft edge): 200 x 0.5 + 50 x 0.5 = **125** -> a smooth mix.

F. Keyboard controls and closing

Code	What it means (in simple words)
`key = cv2.waitKey(1) & 0xFF`	Check which key was pressed (wait 1 millisecond).

Code	What it means (in simple words)
`if key == ord('b'): background = capture_background(cap)`	B = take the background picture.
`elif key == ord('q'): break`	Q = leave the loop.
`cap.release() / cv2.destroyAllWindows()`	Free the camera and close windows.

Common problems and fixes

Problem	Fix
Cloth is still visible	Adjust the HSV limits; add more light.
Parts of the person disappear	Narrow the green range; the person may be wearing green.
Picture looks shifted	The camera moved. Press B again with the cloth out of view.
Shadows remain	Improve lighting; lower the S and V limits.
Webcam does not open	Close other apps using the camera; try VideoCapture(1).

What the teacher says in class
This is not real invisibility. It is a clever picture swap. The computer shows what is behind the cloth, using a photo it took earlier.

Part 3: 3D Modelling and Printing - School Items



Items our students designed and printed: pen holder, phone stand, ruler, gears, house and boat

A 3D printer builds an object by laying down thin layers of melted plastic (called FDM printing). Students first **design** the object on the computer.

Step	What happens
1. Design	Create the 3D model in CAD software (for example a house, pen holder or phone stand).
2. Export STL	Save the model in the .stl file format that printers understand.
3. Slice	Slicing software cuts the model into layers and creates G-code, the printer's instruction list.

Step	What happens
4. Print	The printer melts filament and builds the object layer by layer.
5. Finish	Remove supports, clean and trim the print.

Items students made: lattice pen holder, cube organiser, desk caddy, phone stand, ruler, gears, letter 'A' block, miniature house and a small boat test print.

Key words: layer (one thin slice of plastic), **infill** (the inside pattern that makes a print strong or light), **supports** (temporary plastic that holds up overhangs), **print orientation** (how the object is turned on the bed).

Safety
Supervise all 3D printer use: nozzles and beds are hot and can cause burns. Wear eye protection when trimming prints.

Classroom activity
Design a name tag or a pencil stand. Predict: will it need supports? Which orientation is strongest?

Quick Quiz (with answers)

#	Question	Answer
1	What does 'background = None' mean?	No background picture has been saved yet.
2	Why are 30 frames averaged?	To reduce camera noise.
3	What happens where mask = 1?	The saved background pixel is shown.
4	Why must the camera stay still?	So the saved background lines up with the live picture.
5	What does the Blur do to the mask?	Softens edges for smooth blending.
6	What is G-code?	The instruction file a 3D printer follows, made by slicing software.

CLASS 9

Final Project, Testing, Robots and Drones

Combine everything into one real-time program, test it like an engineer, and meet the spider bot and the drone.

Part 1: The final integrated project

Class 9 joins **camera + color detection + background replacement** and adds professional features: adjustable sliders, FPS counter, a mask window, a before/after comparison and screenshots for the testing report.

Key	What it does
B	Capture the empty background (cloth out of view, camera still)
M	Show or hide the green mask window
C	Compare: original and result side by side
S	Save a screenshot (screenshot_1.png, screenshot_2.png, ...)
Q	Quit

Part 2: The code, part by part

A. Imports and settings

Code	What it means (in simple words)
`import time`	Tool to read the clock. We need it to calculate FPS.
`import cv2 / import numpy as np`	The camera/picture toolbox and the number toolbox.
`CAMERA_INDEX = 0`	Which camera to use. Change to 1 for a second camera.
`FRAME_W, FRAME_H = 640, 480`	Ask for a 640 x 480 picture. Smaller pictures run faster.
`WINDOW = 'Green Cloth - Invisible Cloth'`	The window title, stored once so we can change it in one place.

Why use CAPITAL_LETTER names? They are 'settings' that never change while the program runs. This is a Python habit among programmers.

B. Opening and setting up the camera

Same as before, plus two new lines: `cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_W)` and `cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_H)` ask the camera to use our chosen size. The camera may choose a nearby size if it cannot do exactly that.

C. The Tuning window

Slider	Start	Purpose
H low	30	Lowest hue accepted as green
H high	90	Highest hue accepted as green
S low	30	Lowest color strength; rejects gray and white

Slider	Start	Purpose
V low	15	Lowest brightness; rejects very dark pixels

Now the sliders can be moved **while the project is running**, so we can tune for sunlight, tube light or dim light.

D. Function 1 - capture_background()

Exactly the same as Class 8: take 30 pictures, average them. **Programmer's thinking question:** what if the camera fails and no picture is collected? Then 'collected' is empty and np.mean gives an error. How would you fix it? (Answer: add `if not collected:` check and handle it.)

E. Function 2 - make_green_mask(frame)

Code	What it means (in simple words)
`hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`	Change to HSV.
`lower = np.array([getTrackbarPos(H low), (S low), (V low)])`	Read the three lower sliders to build the lower limit.
`upper = np.array([getTrackbarPos(H high), 255, 255])`	Read the H high slider; S and V upper limits stay 255.
`mask = cv2.inRange(hsv, lower, upper)`	White where green.
`morphologyEx(... MORPH_OPEN ...)`	OPEN: removes small white specks.
`morphologyEx(... MORPH_CLOSE ...)`	CLOSE: fills small black holes inside the white area. New in Class 9!
`cv2.dilate(...)`	Grow the white area slightly.
`return cv2.GaussianBlur(mask, (5, 5), 0)`	Soften the edges and give the finished mask back.

Open and Close are partners
OPEN removes white noise outside. CLOSE fills black holes inside. Think of cleaning a window: wipe off specks outside (open), then fill small scratches (close).

F. Function 3 - replace_with_background()

Same blending formula as Class 8, but now it is a neat function: give it the frame, the background and the mask, and it returns the final picture. **Why use functions?** The main loop becomes short and easy to read, and we can reuse the same function again.

G. State variables

Variable	Purpose
show_mask = False	Mask window starts hidden. M switches it True/False.
compare = False	Side-by-side starts off. C switches it.
prev_time = time.time()	Remembers the time of the previous frame.
fps = 0.0	Starts the FPS number at zero.
shot_count = 0	Counts screenshots saved.

H. The main loop

Code	What it means (in simple words)
`ok, frame = cap.read()`	Take a picture. If it failed, print a message and stop.

Code	What it means (in simple words)
`if background is None:`	No background yet: show the live picture with the instruction text.

`else:`
`    mask = make_green_mask(frame)`
`    output = replace_with_background(frame, background, mask)`
	Background ready: make the mask, then blend. 'output' is the picture we show.

I. FPS - how smooth is our video?

```
now = time.time()
fps = 0.9 * fps + 0.1 * (1.0 / max(now - prev_time, 1e-6))
prev_time = now
```

Part	Simple explanation
now - prev_time	Seconds taken since the last frame.
1.0 / that time	FPS. If one frame takes 0.02 seconds, FPS = 1 / 0.02 = 50.
max(..., 1e-6)	A safety guard so we never divide by zero.
0.9 * old + 0.1 * new	Smoothing: mostly keep the old number, add a little of the new one, so the display does not jump around.
prev_time = now	Remember this moment for the next frame.

`f"FPS: {fps:.1f}"` puts the FPS number into text with **one decimal place** (like 28.4).

J. Display options

Code	What it means (in simple words)
`if compare and background is not None:`	If compare mode is ON and a background exists...
`    cv2.imshow(WINDOW, np.hstack([frame, output]))`	...stack the original and the result side by side (hstack = horizontal stack).
`else: cv2.imshow(WINDOW, output)`	Otherwise show only the result.
`if show_mask and mask is not None:`	Show the mask in its own window...
`try: cv2.destroyWindow('Green Mask')` `except cv2.error: pass`	...or close it. try/except means: 'try this; if an error happens, ignore it and carry on'.

K. Keyboard controls

Code	What it means (in simple words)
`show_mask = not show_mask`	A toggle: True becomes False, False becomes True. Works like a light switch.
`shot_count += 1`	Add one to the screenshot counter (same as shot_count = shot_count + 1).
`name = f'screenshot_{shot_count}.png'`	Build a file name: screenshot_1.png, screenshot_2.png...
`cv2.imwrite(name, output)`	Save the picture to a file in the program folder.

Part 3: Testing like an engineer

Real engineers never trust one test. Test the project in three lighting conditions and fill in this table.

Lighting	Best H low	Best H high	Best S low	Best V low	FPS	Observations
Sunlight						

Lighting	Best H low	Best H high	Best S low	Best V low	FPS	Observations
Tube light						
Dim light						

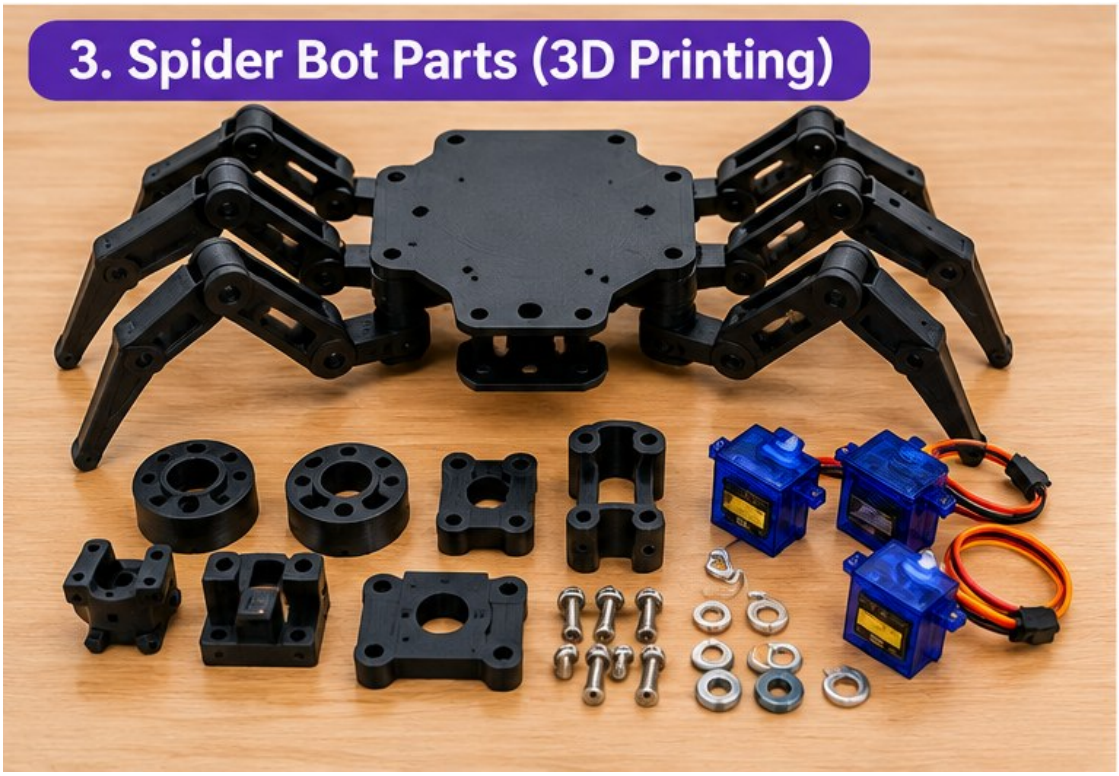
- Place the camera on a stable surface and press **B** with the cloth out of view.
- Bring in the cloth and adjust the sliders until the mask covers only the cloth.
- Press **M** to check the mask, **C** to compare, **S** to save a screenshot.
- Write the best values and the FPS. If the lighting changes a lot, press B again.

Limitations students should understand

- Works best when the cloth is clearly green. Green clothes or objects nearby may also disappear.
- Shadows, reflections and wrinkles reduce accuracy.
- If the camera moves, the background no longer matches.
- FPS depends on the computer and camera. It is an estimate, not an exact benchmark.
- It is a picture trick, not real invisibility.

Rules-based vision vs. trained AI
Our program uses **fixed rules** (HSV ranges). It does not learn. In the TerrAmind AI project, a **trained classifier** looked at a folded green cloth and said 'Green Cloth, Confidence 98.7%, Class: Fabric, Material: Cotton'. That model learned from labelled example photos. Class 9 students should be able to explain the difference: rules are written by us; a trained model learns from examples.

Part 4: Robotics - Spider Bot



Spider bot parts: 3D-printed body plate, legs, brackets and servo motors

A spider bot is a six-legged robot. Its parts are 3D printed in black plastic: a hexagonal body plate, jointed leg segments, servo brackets and round spacers. Three blue micro servo motors, screws, nuts and washers hold everything together.

- **Servo motor:** a motor that turns to an exact angle. Different angles move the leg joints.

- **Degrees of freedom:** the number of directions a joint can move.
- **Tolerance:** a tiny gap so that parts fit together neither too loose nor too tight.
- **Gait:** the pattern of leg movements used to walk.

Step	Action
1	Print the body, legs and brackets.
2	Clean the parts and check the fit.
3	Mount the servos and attach the leg segments with screws.
4	Wire to a controller (such as Arduino) and program the walking motion.

Fix common problems: if joints are loose, change the tolerance in the model and print again. Future idea: add obstacle sensors (like the HC-SR04 from Class 7!) and remote control.

Part 5: Drone assembly



Quadcopter kit: red frame, four motors, flight controller, LiPo battery and radio transmitter

- **Lift and thrust:** spinning propellers push air down, so the drone is pushed up.
- **Pitch, roll and yaw:** tilting forward/back, tilting left/right, and turning left/right.
- **Flight controller:** the drone's brain; it keeps the drone steady.
- **Propeller direction:** two propellers spin clockwise and two anticlockwise, so the drone does not spin around.
- **Binding:** pairing the radio transmitter with the drone.

Step	Action
1	Assemble the frame and mount the motors.
2	Install the flight controller and connect the electronics.
3	Fit propellers in the correct rotation direction.
4	Bind the transmitter, calibrate sensors and test in a safe open area.

Drone and battery safety
Handle LiPo batteries carefully: never puncture, overcharge or leave them charging unattended. Remove propellers during bench testing. Fly only in open, approved areas with a teacher present, and follow local drone rules.

Quick Quiz (with answers)

#	Question	Answer
1	What does FPS stand for?	Frames Per Second.
2	What does MORPH_CLOSE do?	Fills small black holes inside the white mask.
3	What does np.hstack do?	Places two pictures side by side.
4	What does 'show_mask = not show_mask' do?	Toggles between True and False.
5	Why test in different lighting?	Best HSV values change with light.
6	Name one difference between our program and a trained AI model.	Our program follows fixed rules; a trained model learns from labelled examples.
7	Why do drone propellers spin in different directions?	To balance the spinning force so the drone does not rotate.

Teacher's Notes

Suggested lesson plan (about 45 minutes each)

Class	Lesson 1	Lesson 2	Lesson 3
6	Arduino parts, safety, Blink	Traffic light and Running lights	Python color lab
7	Push button and Night lamp	Distance warning	HSV mask with sliders
8	Background capture and blending	Cleaning the mask, fixing problems	3D design and printing
9	Final program walkthrough	Testing in 3 lightings, report	Spider bot or drone demo

Teaching tips

- Work in groups of 3 to 4. Give roles: builder, coder, tester, recorder.
- Let students **predict before they run**: 'What will happen if I change this number?'
- Encourage mistakes. Errors are clues. Read the error message aloud together.
- Always revise the previous class's idea in the first 5 minutes.
- The camera code uses cv2.CAP_DSHOW, which is for Windows. On Mac or Linux, use cv2.VideoCapture(0).
- Install first: `pip install opencv-python numpy`

Assessment ideas

- Practical: wire and run a project without help (Classes 6-7).
- Explain-it test: a student explains the blending formula using numbers (Class 8).
- Testing report with the lighting table and screenshots (Class 9).
- Documentation: a short project diary with photos, written by the group.

Skills developed

Project	Technology	Key skills
Green Cloth	Artificial Intelligence / Computer Vision	Data labelling, model testing, interpreting confidence, HSV masks
School Items	3D Modelling and Printing	CAD design, slicing, printer operation
Spider Bot Parts	3D Printing and Robotics	Mechanical design, servo control, assembly
Drone Assembly	Drone Technology	Electronics, calibration, flight control, safety
Arduino projects	Electronics and Coding	Inputs, outputs, sensors, logic, debugging

Challenges and solutions

- **Print warping or failed layers:** level the bed, adjust temperature and use proper adhesion.
- **Loose joints in the robot:** adjust tolerances in the model and reprint.
- **Unstable drone flight:** recheck motor direction, balance propellers and recalibrate.
- **AI misclassification:** add more varied training images.
- **LED does not light:** check the long leg (+), the resistor and the pin number in the code.

Future scope

- Train the AI model to recognise many fabrics and colours.
- Multi-colour and functional-assembly 3D prints.
- Add obstacle sensors and remote control to the spider bot.
- Autonomous drone missions and camera payloads.

Build with curiosity. Learn from mistakes. Work in teams. Create something amazing!